

Hierarchical Planning with Latent World Models

Wancong Zhang^{1,2,*}, Basile Terver¹, Artem Zhohus^{1,3}, Soham Chitnis², Harsh Sutaria², Mido Assran¹, Randall Balestriero^{1,4}, Amir Bar¹, Adrien Bardes¹, Yann LeCun^{2,†}, Nicolas Ballas^{1,†}

¹FAIR at Meta, ²New York University, ³Mila - Québec AI Institute, ⁴Brown University

*Work done at Meta, †Joint last author

Model predictive control (MPC) with learned world models has emerged as a promising paradigm for embodied control, particularly for its ability to generalize zero-shot when deployed in new environments. However, learned world models often struggle with long-horizon control due to the accumulation of prediction errors and the exponentially growing search space. In this work, we address these challenges by learning latent world models at multiple temporal scales and performing hierarchical planning across these scales, enabling long-horizon reasoning while substantially reducing inference-time planning complexity. Our approach serves as a plug-in planning abstraction that applies across diverse latent world-model architectures and domains. We demonstrate that this hierarchical approach enables zero-shot control on real-world non-greedy robotic tasks, achieving a 70% success rate on pick-&-place using only a final goal specification, compared to 0% for a single-level world model. In addition, across physics-based simulated environments including push manipulation and maze navigation, hierarchical planning achieves higher success while requiring up to 3× less planning-time compute.

Date: 2026-04-06

Correspondence: Wancong Zhang at wz1232@nyu.edu

Code: https://github.com/kevinghst/HWM_PLDM

Website: <https://kevinghst.github.io/HWM/>



1 Introduction

Planning with learned world models has emerged as a powerful paradigm for embodied decision making. By learning predictive models of environment dynamics from experience and using them to simulate future outcomes, world models enable agents to reason about the long-term consequences of actions (Sutton, 1981; Deisenroth and Rasmussen, 2011). Recent advances in representation learning and latent-space modeling have improved the scalability of this approach, making it possible to train world models directly from high-dimensional observations such as images and videos (Ha and Schmidhuber, 2018; Hafner et al., 2023; Hansen et al., 2023). More recently, self-supervised learning has enabled training world models from large collections of unlabeled, task-agnostic offline trajectories, allowing zero-shot deployment in downstream robotic control and navigation (Assran et al., 2025; Sobal et al., 2025; Bar et al., 2025; Zhou et al., 2024; Terver et al., 2026b). Despite this progress, learned world models continue to struggle with long-horizon control due to two fundamental challenges: (i) prediction errors compound over time, leading to increasingly inaccurate rollouts (Talvitie, 2014; Janner et al., 2019), and (ii) even under a perfect model, planning becomes computationally expensive as the search space grows exponentially with horizon (Ichter et al., 2020).

A classic approach to long-horizon control is to introduce *hierarchy* into the decision-making structure. By reasoning at multiple temporal scales, agents can plan coarsely over long horizons while retaining fine-grained control at short horizons. This intuition underlies a large body of work on temporal abstraction in reinforcement learning, including options, skills, and hierarchical policies (Sutton et al., 1999; Bacon et al., 2017; Hafner et al., 2022; Park et al., 2023), as well as hierarchical world models that capture dynamics at multiple timescales (Gumbsch et al., 2023; Chitnis et al., 2024; Schiewer et al., 2024; Hansen et al., 2024). In parallel, the optimal control literature has explored hierarchical model predictive control (MPC) for multi-task settings (Fang et al., 2019; Li et al., 2021; Kögel et al., 2025). However, these approaches remain limited in complementary ways. Reinforcement learning methods typically rely on task-specific rewards, which restrict their applicability in zero-shot settings; even goal-conditioned and zero-shot RL methods (Park et al., 2023, 2024b) still struggle

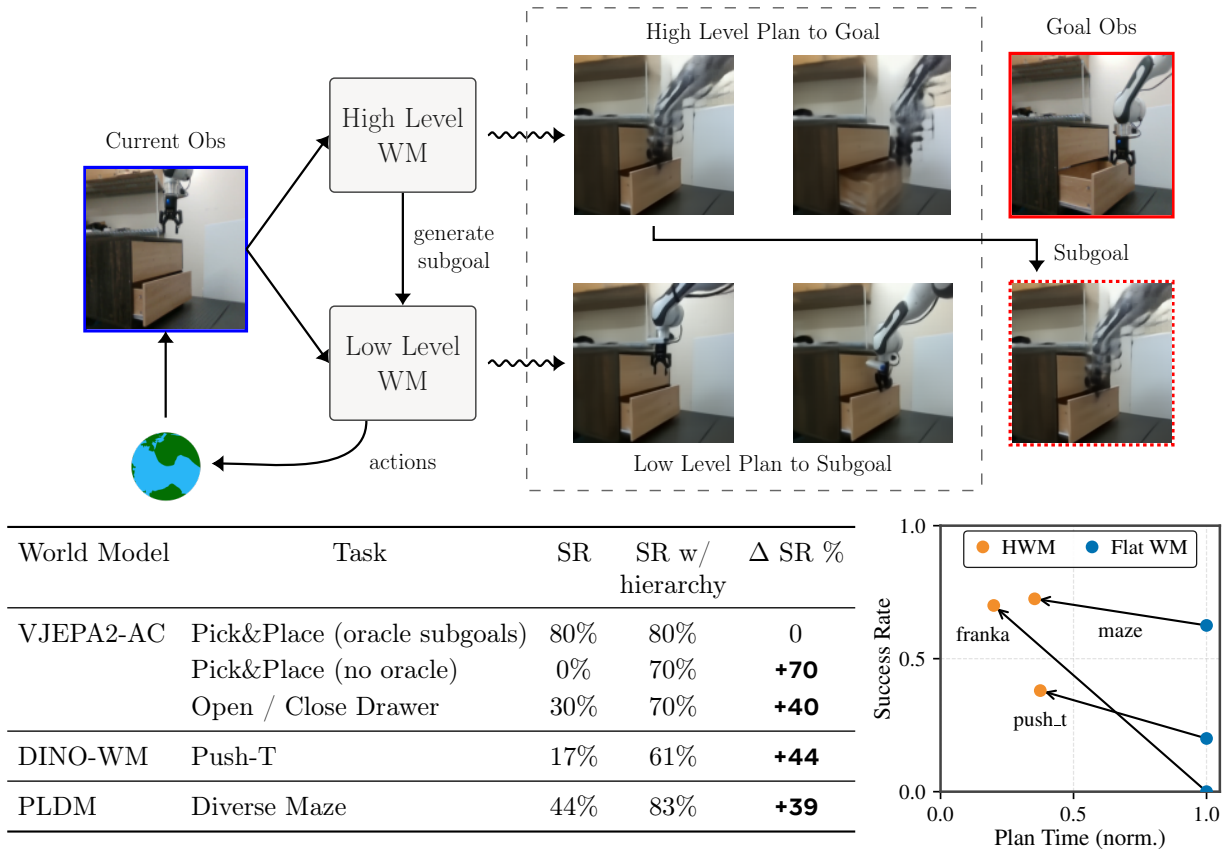


Figure 1 Top: Hierarchical planning in latent space. A high-level planner optimizes macro-actions using a long-horizon world model to reach the goal; the first predicted latent state serves as a subgoal for a low-level planner, which optimizes primitive actions with a short-horizon world model. Solid borders denote ground-truth observations; others are decoder reconstructions shown for interpretability only. **Bottom Left: Empirical coverage.** Hierarchical planning improves success on non-greedy, long-horizon tasks across multiple latent world models. Δ SR denotes the absolute success-rate gain from hierarchy. *Oracle subgoals* use externally provided subgoal images; otherwise, planning relies only on the final goal. **Bottom Right: Planning efficiency.** Success rate versus normalized planning time for flat and hierarchical planners; hierarchical planning matches or exceeds success with $\approx 3\times$ lower planning time. Highlighted HWM results prioritize planning efficiency rather than peak performance.

to generalize to environments and tasks beyond the training distribution (Sobal et al., 2025). Conversely, existing hierarchical MPC methods are not task-specific but are largely restricted to low-dimensional state spaces, hand-engineered representations, or known dynamics, limiting their applicability to high-dimensional observation spaces such as raw pixels. As a result, it remains unclear how to realize hierarchical planning for zero-shot control directly on learned world models from high-dimensional observations.

In this work, we propose *Hierarchical Planning with Latent World Models* (HWM), a framework for zero-shot hierarchical model predictive control using learned latent world models. Rather than learning hierarchical policies, HWM introduces temporal hierarchy directly at inference time by planning over multiple timescales on task-agnostic world models trained via next-latent prediction. Crucially, our approach is defined by how the hierarchical levels interact. HWM learns world models operating at different temporal resolutions within a shared latent space, enabling a direct coupling between levels: high-level predictions act as subgoals for low-level MPC via latent-state matching. This removes the need for inverse models, skill learning, or goal-conditioned policies, and allows hierarchical planning to operate entirely on pretrained world models. To support efficient long-horizon planning, we further introduce a learned action encoder that compresses sequences of primitive actions between waypoint states into latent macro-actions. This reduces the dimensionality of the high-level search space and makes planning more tractable. By decoupling long-horizon reasoning from fine-grained control, HWM mitigates error accumulation and reduces overall planning complexity.

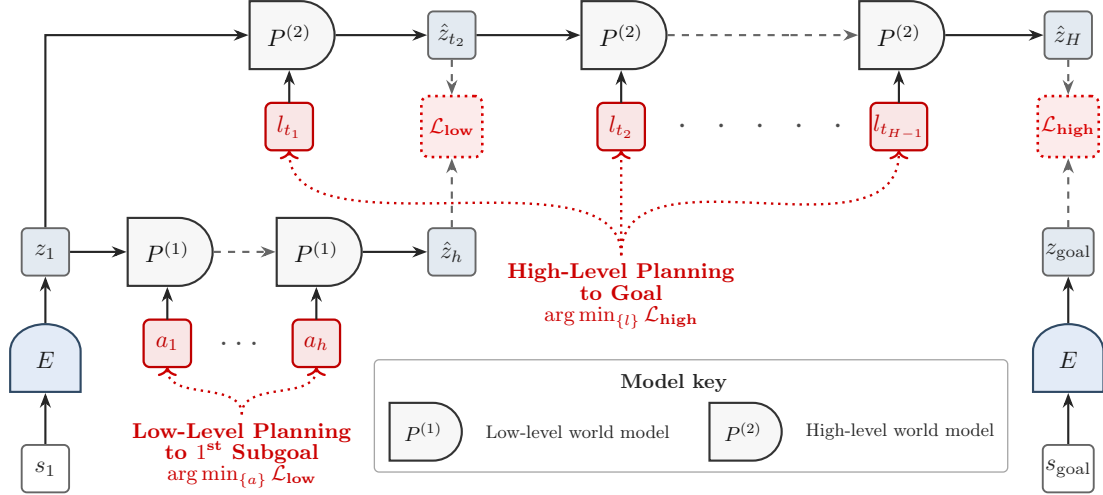


Figure 2 Hierarchical planning in latent space. A high-level planner optimizes macro actions using a long-horizon latent world model to reach the final goal embedding. The resulting first predicted latent state serves as a subgoal for low-level planning, where a short-horizon world model optimizes primitive actions to reach this subgoal.

We summarize our contributions as follows:

- We introduce HWM, a hierarchical MPC framework that couples multiple learned world models via a shared latent space, enabling direct subgoal transfer across levels without hierarchical policies, skill learning, or task-specific rewards.
- We show that HWM unlocks a new capability for zero-shot world-model planning: solving non-greedy real-robot tasks from visual inputs, where success requires temporarily moving away from the goal. HWM achieves 70% success on Franka pick-&-place from a single goal image, compared to 0% for VJEP2-AC, and also outperforms strong vision-language-action models trained on $\sim 77\times$ more robotic interaction data.
- We demonstrate that HWM is a modular, model-agnostic planning abstraction, consistently improving zero-shot performance across diverse latent world models, including VJEP2-AC (robot manipulation), DINO-WM (push manipulation), and PLDM (maze navigation).
- We show that hierarchical planning improves both performance and efficiency on long-horizon tasks, achieving substantial success-rate gains (up to +44% and +39%) while reducing inference-time planning cost by up to $4\times$ compared to single-level planners.

2 Hierarchical Planning with Latent World Models

2.1 Problem Setting

We consider a Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mu, p)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mu \in \mathcal{P}(\mathcal{S})$ is the initial state distribution, and $p : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$ denotes the transition dynamics. We work in the offline setting with access to a dataset $\mathcal{D}$ of state-action trajectories $\tau = (s_1, a_1, s_2, \dots, a_{T-1}, s_T)$. Our objective is to use $\mathcal{D}$ to enable zero-shot control across a variety of goal-conditioned tasks. Rather than learning an explicit parametric policy, we learn world models to approximate the true transition dynamics p , and select actions at inference time via model predictive control (MPC), which induces a goal-conditioned policy. We consider goal-reaching tasks at evaluation time, where the agent is provided with a goal state s_g and succeeds by reaching it. In this setting, the task space $\mathcal{Z}$ coincides with the state space $\mathcal{S}$.

2.2 Latent World Models

A *world model* is a learned dynamical model that approximates how an environment evolves in response to an agent’s actions, estimating transition dynamics of the form $p(s_{t+1} | s_t, a_t)$. In high-dimensional settings, world models are typically learned in a *latent space* to improve efficiency and robustness. An encoder maps

observations s_t to latent states z_t , and a latent world model predicts future latents conditioned on actions, $p(z_{t+1} | z_t, a_t)$. Recent work has shown that latent world models learned from reward-free, task-agnostic offline data can support zero-shot, goal-conditioned control in downstream planning tasks (Assran et al., 2025; Sobal et al., 2025; Bar et al., 2025; Zhou et al., 2024; Terver et al., 2026b). However, planning directly with such models using flat MPC becomes increasingly brittle as task horizons grow, due to compounding prediction errors and the expanding search space.

2.3 Top-Down Hierarchical Planning

Latent world models naturally support hierarchical planning, as different levels of abstraction can reason over different temporal horizons within a shared predictive framework. By planning at an appropriate temporal resolution, hierarchical methods avoid direct optimization over long sequences of primitive actions while retaining fine-grained control.

In this work, we adopt a *top-down* hierarchical planning strategy that operates entirely in a learned latent space (Fig. 2). At a high level, the agent plans abstract latent actions toward a final goal using a long-horizon world model. The resulting predicted latent states serve as intermediate subgoals. A low-level planner then optimizes short-horizon sequences of primitive actions to reach the next subgoal, repeating this process in a receding-horizon MPC manner.

We now describe the latent representations and world models used by the hierarchical planner. Let E denote an encoder that maps observations s_t to latent states $z_t = E(s_t)$. We assume access to two learned latent world models operating at different temporal resolutions: **(i)** a low-level model $P^{(1)}(z_{t+1} | z_t, a_t)$ that predicts short-horizon transitions conditioned on primitive actions, and **(ii)** a high-level model $P^{(2)}(z_{t+h} | z_t, l_t)$ that predicts long-horizon transitions conditioned on latent macro-actions l_t .

The macro-actions l_t are produced by a learned action encoder that summarizes temporally extended sequences of low-level actions. Unlike fixed-stride temporal abstraction methods, we do not assume a fixed high-level horizon h , allowing each high-level transition to correspond to a variable-length segment of low-level execution.

High-Level Planning. Given an initial observation s_1 and a goal observation s_g , we encode them into latent space as $z_1 = E(s_1)$ and $z_g = E(s_g)$. At the high level, we plan over a sequence of H latent actions $\hat{l}_{1:H}$ by minimizing a goal-conditioned energy function in latent space:

$$\mathcal{E}_2(\hat{l}_{1:H}; z_1, z_g) \triangleq \|z_g - P^{(2)}(\hat{l}_{1:H}; z_1)\|_1$$

Here, $P^{(2)}(\hat{l}_{1:H}; z_1)$ denotes the final latent state obtained by autoregressively unrolling the world model $P^{(2)}$ starting from z_1 with latent actions $\hat{l}_{1:H}$. The optimal latent action sequence is given by: $l_{1:H}^* = \arg \min_{\hat{l}_{1:H}} \mathcal{E}_2(\hat{l}_{1:H}; z_1, z_g)$.

Unrolling the optimized latent plan yields a sequence of intermediate latent subgoals at the high-level temporal resolution: $\tilde{z}_i \triangleq P^{(2)}(l_{1:i}^*; z_1)$, $i = 1, \dots, H$.

Low-Level Planning. At execution time, the agent plans primitive actions to reach the first latent subgoal $\tilde{z}_1$. Starting from the current latent state z_1 , we define a low-level energy function over a horizon of h steps:

$$\mathcal{E}_1(\hat{a}_{1:h}; z_1, \tilde{z}_1) \triangleq \|\tilde{z}_1 - P^{(1)}(\hat{a}_{1:h}; z_1)\|_1$$

The optimal low-level action sequence is obtained via $a_{1:h}^* = \arg \min_{\hat{a}_{1:h}} \mathcal{E}_1(\hat{a}_{1:h}; z_1, \tilde{z}_1)$, which is fed into the controller. The agent re-plans every k interactions using the hierarchical planner.

2.4 Low-Level Latent World Model

The low-level latent world model $P_\theta^{(1)}$ predicts short-horizon transitions in latent space conditioned on primitive actions. Given an observation s_t , we encode it into a latent representation $z_t = E(s_t)$ and use $P_\theta^{(1)}$ to model $p(z_{t+1} | z_t, a_t)$. The model is trained using standard latent world-model objectives, including teacher-forcing and multi-step autoregressive rollout losses, following prior work on JEPA-style predictive models (appendix B.1). In our experiments, we reuse the training recipes and architectures of the underlying

backbone world models (VJEPA2-AC, DINO-WM, and PLDM), ensuring that any performance gains are attributable to hierarchical planning rather than changes in the low-level dynamics model.

2.5 High-Level Latent World Model

We describe how to train a high level latent world model $P_\phi^{(2)}$ that predicts over longer time horizons (Fig. 3). Given a trajectory $\tau = (s_1, a_1, s_2, a_2, \dots, s_T)$, we first choose N waypoint indices such that $N < T$ and $1 = t_1 < t_2 < t_3 < \dots < t_{N-1} < t_N$. Then the waypoint states are $(s_{t_1}, s_{t_2}, \dots, s_{t_{N-1}}, s_{t_N})$. Each high-level transition consists of a waypoint s_{t_k} , the next waypoint $s_{t_{k+1}}$, and the action subsequence between them. Formally, the k^{th} transition is $(s_{t_k}, (a_{t_k:t_{k+1}}), s_{t_{k+1}})$. The high level world model makes predictions in the same latent space as the low-level world model, and uses the shared encoder E to transform the waypoint states $(s_{t_k})_{k \in [N]}$ into latent features $(z_{t_k})_{k \in [N]}$. The action encoder A_ψ encodes the action subsequences between waypoints $(a_{t_k:t_{k+1}})_{k \in [N]}$ into macro latent actions $(l_{t_k})_{k \in [N]}$. We feed the interleaved sequence of waypoint latent actions and features $(l_{t_k}, z_{t_k})_{k \in [N]}$ into the world model $P_\phi^{(2)}$ to predict the next waypoint representation predictions $(\hat{z}_{t_{k+1}})_{k \in [N]}$. The teacher-forcing loss is:

$$\begin{aligned} \hat{z}_{t_{k+1}} &:= P_\phi^{(2)}\left((l_{t_i}, z_{t_i})_{i \leq k}\right) \\ \mathcal{L}_{\text{tf}}(\phi, \psi) &:= \frac{1}{N} \sum_{k=1}^N \|\hat{z}_{t_{k+1}} - z_{t_{k+1}}\|_1 \end{aligned} \quad (1)$$

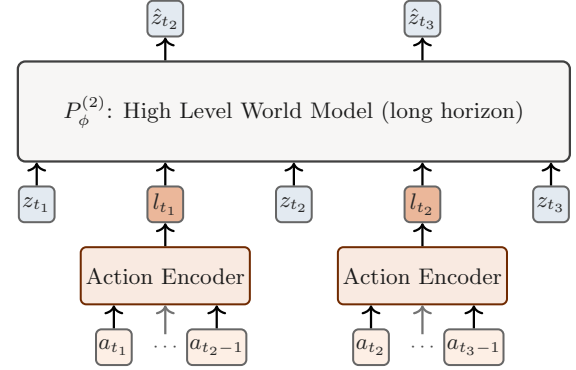


Figure 3 The high-level world model captures long-horizon dynamics using latent actions encoded from chunks of low-level actions. It is causal and trained with a latent prediction loss (Eq. (1)).

3 Empirical Evaluation of Hierarchical Planning Across World Models and Tasks

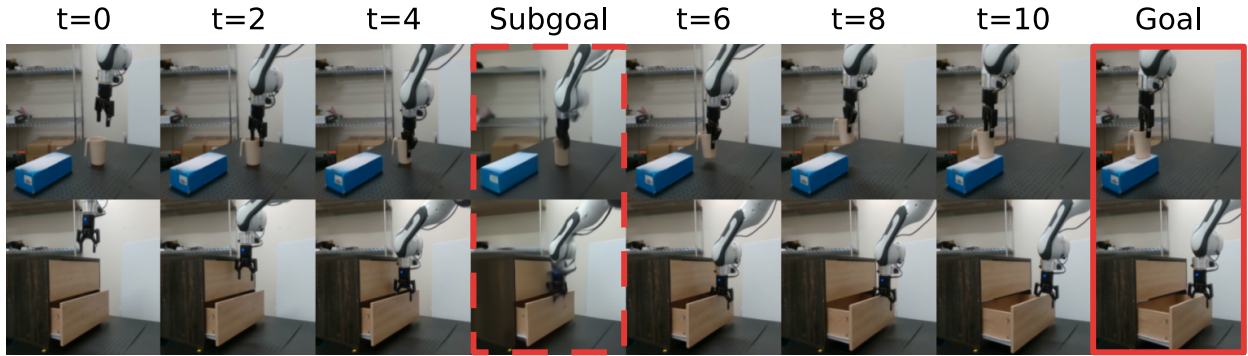
We evaluate whether hierarchical planning (HWM) enables zero-shot long-horizon control in regimes where planning with a single-level learned world model fails. We apply HWM to three representative latent world-model architectures—VJEPA2-AC (Assran et al., 2025), PLDM (Sobal et al., 2025), and DINO-WM (Zhou et al., 2024)—and, for each, consider a benchmark domain from the original work while substantially increasing task horizon and complexity. For VJEPA2-AC, we study real-world pick-&-place and drawer manipulation on a Franka robot arm without explicit subgoals. For DINO-WM, we consider the Push-T task with task horizons extended from 25 to 75 timesteps. For PLDM, we evaluate point-maze navigation with train-test layout mismatch while quadrupling map size. We summarize our main findings as follows:

1. **Zero-shot non-greedy control:** Pick-&-place success improves from 0% to 70% on Franka with VJEPA2-AC.
2. **Long-horizon reasoning:** Push-T success increases from 17% to 61% with DINO-WM.
3. **OOD generalization:** HWM improves PLDM performance on navigation in larger, unseen mazes.
4. **Compute efficiency:** Comparable or better planning relative to single-level planners with $3 \times$ less compute.

3.1 Franka Arm with Robotiq Gripper

We evaluate HWM on real-world multi-stage manipulation with a Franka arm equipped with a Robotiq gripper, resembling the Droid setup. We run pick-&-place and drawer opening/closing. These tasks require *non-greedy* behavior, where successful execution depends on intermediate motions that temporarily increase distance to the goal—a regime in which single-level planners typically fail.

Platform and Task. We follow the VJEPA2-AC deployment setup and evaluate on real-world pick-&-place and drawer manipulation using a 7-DoF Franka Emika Panda arm with a two-finger gripper. Pick-&-place



(a) Executions on pick-&-place and drawer manipulation with a dual-gripper Franka arm. Dotted red columns indicate subgoals inferred by the high-level planner and decoded for visualization.



(b) Executions on Push-T task. Dotted red columns indicate subgoals inferred by the high-level planner and decoded for visualization.

includes 10 start-goal pairs across two objects (cup and box), while drawer manipulation covers 7 opening and closing tasks (see [appendix G](#)). Success is defined as reaching the goal.

Baselines. **VJEP2-AC** ([Assran et al., 2025](#)) is a single-level, world-model-based planner. Because we use the same architecture and training data for our low-level world model, this comparison isolates the effect of hierarchical planning. We additionally evaluate three vision-language-action models (VLA) baselines **Octo**, π_0 -FAST-DROID, and $\pi_{0.5}$ -DROID (see [appendix J.1](#) for details).

Data. World models are trained on approximately 130 hours of unlabeled real-robot manipulation data from DROID ([Khazatsky et al., 2024](#)) and RoboSet ([Kumar et al., 2023](#)). Observations include RGB images and end-effector proprioception; actions correspond to end-effector delta poses (see [appendix B.2](#) for details).

Hierarchical Planning Setup. The agent is provided with the current and goal observations, each specified by RGB images and end-effector poses. Agent’s actions are end-effector delta poses that are mapped to joints delta poses via inverse kinematics. For each action, the corresponding joints delta is executed in a blocking fashion. The high-level planner optimizes over latent actions of dimension 4, using 3000 CEM samples with a prediction horizon of 2, while the low-level planner uses 800 CEM samples with a prediction horizon of 2. The agent replans every $k = 1$ step in standard MPC manner. Planning rollouts are executed in parallel across GPUs, making the approach computationally efficient even with large numbers of trajectory samples. Full CEM hyperparameters are provided in [appendix C](#).

World Model Architectures. We follow the VJEP2-AC training recipe for the low-level world model (see [appendix B.2](#)). To train the high-level world model, We use $N = 3$ waypoint states sampled from trajectory segments spanning up to 4 seconds, with the middle waypoint chosen uniformly at random. Latent actions are encoded from the intervening low-level actions using a transformer-based action encoder, with the CLS token projected to the latent action space. For visualization, we decode latent representations into RGB images using a ViT decoder; these reconstructions are used solely for qualitative analysis and are not involved in planning or training (details in [appendix E](#)).

Main Results. We report the findings in [table 1](#). Prior work with VJEP2-AC ([Assran et al., 2025](#)) has shown strong zero-shot generalization to novel environments for simple, greedy tasks such as grasping or reaching. Consistent with this, on the drawer benchmark the single-level planner succeeds on 2/7 tasks that admit

linear push or pull motions. However, for more complex non-greedy tasks—such as pick-&-place and the remaining drawer variants—the single-level planner can only succeed when the task is decomposed into a sequence of greedy subtasks via manually provided subgoal images; without such supervision, it fails to solve these tasks end-to-end.

In contrast, hierarchical planning enables end-to-end solution of both pick-&-place and drawer tasks using only a final goal image. It generalizes in a zero-shot manner to novel environments and unseen objects, achieving a 70% success rate for pick-&-place cup and 60% for pick-&-place box, and is capable of solving non-greedy drawer tasks that require multi-stage motion (e.g., moving down before translating laterally). Example executions are shown in Fig. 4a and appendix H. Notably, HWM consistently outperforms strong vision-language-action (VLA) baselines, despite these models being pretrained on $\sim 77\times$ more robotic interaction datasets. Failure cases primarily arise from perceptual imprecision (e.g., depth estimation errors) or occasional near-miss executions due to lack of joint optimization between high- and low-level planners.

Takeaway: HWM enables zero-shot, non-greedy manipulation on a real robot from a single goal image, where flat world-model planners fail.

3.2 Push-T

We instantiate HWM on top of DINO-WM (Zhou et al., 2024) and evaluate it on the Push-T environment introduced by Chi et al. (2025) and adapted by DINO-WM to a goal-conditioned setting. The task requires fine-grained control to push a T-shaped object to match a goal configuration sampled from a validation trajectory. To stress-test long-horizon planning, we consider extended-horizon settings with start-goal pairs separated by up to $d = 75$ timesteps, exceeding the $d = 25$ timesteps used in the original DINO-WM evaluation. All models are trained on the offline Push-T dataset released with DINO-WM (details in appendix B.3).

Baselines. We compare against (i) DINO-WM, (ii) offline goal-conditioned RL methods GCIQL and HIQL (Kostrikov et al., 2021; Park et al., 2023), (iii) HILP (Park et al., 2024b), a zero-shot unsupervised RL method.

Hierarchical Planning Setup. Hierarchical planning is performed using CEM at two temporal scales, with a high-level planner optimizing latent actions and a low-level planner refining primitive actions in a receding-horizon manner. Full CEM hyperparameters are provided in appendix C.

Main Results. As shown in table 2, hierarchical planning consistently outperforms DINO-WM as task difficulty increases. In particular, while the flat planner’s success rate drops sharply as the task horizon grows, the hierarchical planner remains substantially more robust at longer horizons. Beyond absolute performance, hierarchical planning also exhibits a more favorable compute-performance trade-off. For a fixed task setting, the hierarchical planner achieves higher success rates while requiring $3\times$ less compute per planning step compared to the flat planner (Fig. 5). These results highlight the advantage of planning over temporally abstract latent actions, which reduces optimization complexity while preserving effective long-horizon control. Moreover, policy baselines (GCIQL,

Method	P&P (Subgoals)		P&P		Drawer
	Cup	Box	Cup	Box	
Octo	20%	10%	0%	0%	43%
π_0 -FAST-DROID	–	–	52%	18%	–
$\pi_{0.5}$ -DROID	–	–	68%	36%	–
VJEP2-AC	80%	80%	0%	0%	30%
VJEP2-AC (hierarchy)	80%	80%	70%	60%	70%

Table 1 Zero-shot robotic manipulation with Franka dual-gripper, including pick-&-place and drawer manipulation. Flat planner succeeds only when tasks are decomposed into greedy subtasks via manual subgoals, whereas HWM enables end-to-end execution from a final goal image.

Method	$d = 25$	$d = 50$	$d = 75$
GCIQL	40%	25%	7.5%
HIQL	55%	30%	20%
HILP	25%	13%	0%
DINO-WM	84%	55%	17%
DINO-WM (hierarchy)	89%	78%	61%

Table 2 Push-T Performance Across Task Horizons. In each trial, a random start-goal pair d timesteps apart is sampled from a validation trajectory.

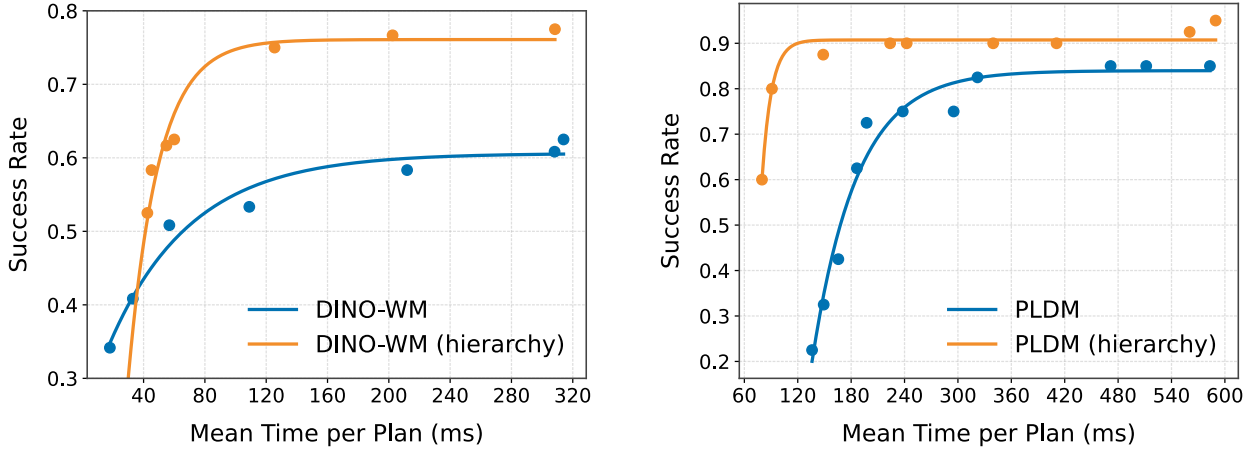


Figure 5 Success rate vs. test-time compute. We vary the planning budget by sweeping CEM/MPPI samples and iterations (see [appendix D](#)). Hierarchical planning achieves higher success while using up to $3\times$ less test-time compute than flat planners. **Left:** Push-T success rate vs. planning time ($d = 50$). **Right:** Diverse maze success rate vs. planning time ($D \in [9, 12]$).

HIQL, HILP) exhibit a pronounced drop in success at longer horizons, suggesting limited robustness to long-horizon generalization without explicit planning. See [Fig. 4b](#) for examples of executions and subgoals.

Takeaway: HWM improves long-horizon control while substantially reducing inference-time compute.

3.3 Diverse Maze

We instantiate HWM on top of PLDM ([Sobal et al., 2025](#)) and evaluate it on Diverse Maze navigation, focusing on zero-shot generalization to larger, unseen layouts that require long-horizon planning.

Platform and Task. We build on the MuJoCo PointMaze environment ([Todorov et al., 2012](#)), using top-down RGB renderings as input. Maze layouts are randomly generated on a 10×10 grid with connected free space; models are trained on 25 layouts and evaluated on 20 held-out layouts. Start and goal locations are sampled uniformly with grid-distance separation H , defining easy ($D \in [5, 8]$), medium ($D \in [9, 12]$), and hard ($D \in [13, 16]$) regimes. Dataset and training details are provided in [appendix B.4](#).

Planning and Baselines. Hierarchical planning uses CEM at two temporal scales (details in [appendix B.4](#)). We compare against the flat PLDM planner, goal-conditioned RL baselines GCIQL and HIQL ([Park et al., 2023](#)), and the zero-shot RL method HILP ([Park et al., 2024b](#)).

Main Results. As shown in [table 3](#), the performance gap between HWM and PLDM widens as task horizon increases. Even when the flat planner performs reasonably well, it requires substantially higher test-time compute to match HWM performance ([Fig. 5](#)). In contrast, HWM achieves higher success with $4\times$ less compute. Notably, both PLDM variants—with and without hierarchy—outperform goal-conditioned and zero-shot RL policy baselines when generalizing to out-of-distribution maze layouts, highlighting the robustness of model-based planning to distribution shift in environment geometry. See [Fig. 10](#) for visualizations of high-level plans.

Method	$D \in [5, 8]$	$D \in [9, 12]$	$D \in [13, 16]$
GCIQL	85%	40%	33%
HIQL	88%	73%	48%
HILP	48%	20%	10%
PLDM	100%	63%	44%
PLDM (hierarchy)	100%	95%	83%

Table 3 Diverse Maze Performance Across Task Horizons. Agents are evaluated zero-shot on navigation tasks in test maps with layouts unseen during training. Start and goal positions are randomly sampled at varying grid distances.

Takeaway: HWM is a modular, model-agnostic extension that consistently improves zero-shot planning across diverse latent world models.

4 Analysis of Hierarchical Planning

We conduct ablation experiments to analyze factors that influence hierarchical planning performance. All models in this section use a VJEPa2 backbone trained on real-world robotic datasets (training details in [section 3.1](#)).

4.1 Do latent actions help with high level planning?

We examine whether training the high-level world model benefits from encoding the full low-level action subsequence between waypoint states, as opposed to representing high-level actions solely as the net end-effector displacement between waypoints. We do not consider using raw action sequences directly, as this would resemble flat planning and would not reduce the action search space for planning.

As shown in [table 4](#), compared to a high-level world model trained with delta-pose actions, a model trained with latent actions produces plans that more closely align with expert high-level behavior (see [appendix I](#) for evaluation protocol). This improvement arises because waypoint states may be separated by long temporal windows, during which the underlying action sequence can be highly non-greedy (e.g., first moving upward, then downward). A delta-pose representation collapses such trajectories into a single displacement, discarding information critical for predicting feasible transitions, whereas a learned latent action can compactly encode the structure of the full action sequence.

High Level Action	Action Cos $\uparrow$	Action L1 $\downarrow$
Delta Pose	0.80 ± 0.02	0.088 ± 0.005
Latent Action	0.88 ± 0.03	0.080 ± 0.002

Table 4 Action alignment to expert trajectories for high-level models using delta-pose actions vs learned latent actions (mean $\pm$ SE). We report cosine similarity ($\uparrow$) and L1 distance ($\downarrow$) between inferred and expert behavior.

4.2 Do High-Level World Models Improve Long Horizon Prediction?

A key challenge in long-horizon planning with learned world models is error accumulation during autoregressive rollouts, where small one-step errors compound over time. We hypothesize that models trained at longer temporal scales yield more accurate long-horizon predictions by reducing the number of autoregressive steps required. To test this, we condition both low-level and high-level world models—trained on the DROID dataset—on held-out initial observations and predict future states up to 2 seconds ahead, measuring ℓ_1 error to the ground-truth future latent state. Predicting 2 seconds ahead requires up to 16 autoregressive steps for the low-level model, whereas the high-level model produces a single-step prediction. As shown in [Fig. 6](#), the low-level model is more accurate for short horizons (≤ 1 s), while the high-level model achieves lower error for longer horizons (≥ 1.5 s), supporting a hierarchical strategy in which high-level planning provides long-term guidance and low-level planning handles short-term precision.

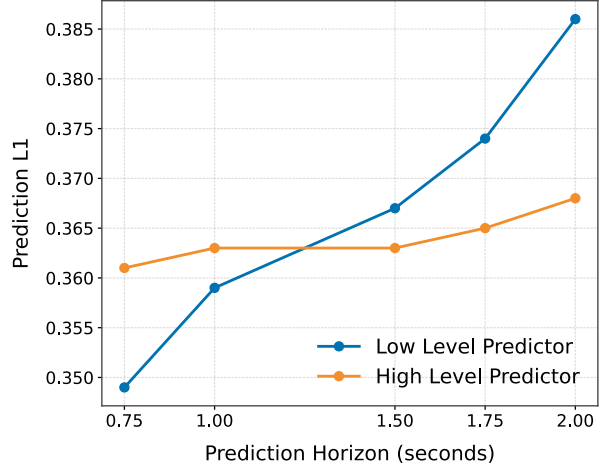


Figure 6 Prediction error (L_1) as a function of prediction horizon. For short horizons (≤ 1 s), the low-level world model is more accurate. For longer horizons (≥ 1.5 s), a single-step prediction from the high-level world model outperforms autoregressive rollouts of the low-level model, reflecting reduced error accumulation.

4.3 How to generate good subgoals?

For hierarchical planning to be effective, the high-level planner must propose subgoals that lie on a feasible path to the final goal and are reachable by the low-level planner. High-level plans therefore fall into three categories: invalid plans, valid plans with unreachable subgoals, and valid plans with reachable subgoals.

As shown in [Fig. 7](#), when the latent action space has sufficient capacity (≥ 4 dimensions), the high-level

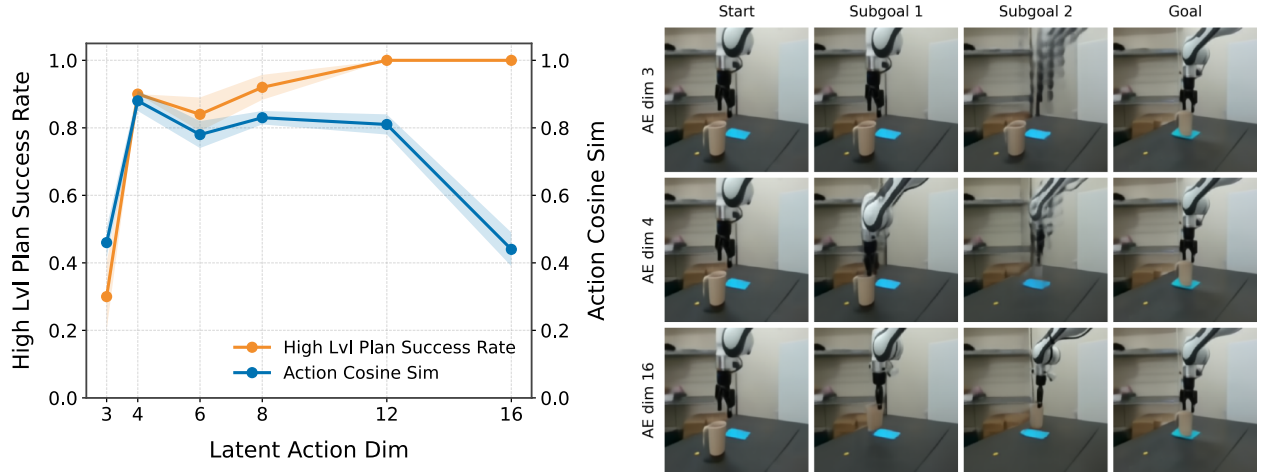


Figure 7 Effect of latent action dimension on hierarchical planning. **Left:** Performance as a function of latent action dimension d . When the latent space has very low capacity, the high-level planner fails to produce valid plans. As d increases, plan success improves; however, beyond a moderate dimensionality, the proposed subgoals become increasingly difficult for the low-level planner to execute, as indicated by a drop in action cosine similarity to expert behavior. **Right:** Qualitative rollouts illustrating this trade-off. Moderate latent dimensionality biases the planner toward proposing reachable, greedy subgoals, yielding the best overall performance.

planner typically produces valid plans. However, these subgoals are not always reachable by the low-level planner, as they may require non-greedy action sequences. Restricting the latent-action dimensionality biases the planner toward proposing subgoals that are achievable with greedy behavior. This suggests an optimal regime in which the latent space is expressive enough to encode useful trajectories, but not so expressive that it enables subgoals requiring complex non-greedy execution. Empirically, a latent action dimension of 4 strikes this balance for the Franka tasks.

Notably, reconstruction fidelity is not tightly correlated with latent-action dimensionality. Lower-dimensional latent spaces yield noisier predictions, reflected in higher ℓ_1 error and blurrier reconstructions (Fig. 7), as expected under stronger compression. Nevertheless, these predictions often preserve coarse semantic structure, such as contact events or motion direction, which is sufficient for hierarchical planning despite reduced visual precision.

5 Related Works

World Models and Planning. Learning predictive models of environment dynamics is a core idea in model-based reinforcement learning (Sutton, 1981), with early work focusing on state-space dynamics for planning and policy optimization (Deisenroth and Rasmussen, 2011; Sutton, 1991). More recent approaches learn dynamics from high-dimensional observations, either directly in pixel space or in a learned latent space (Ebert et al., 2017; Ha and Schmidhuber, 2018; Hafner et al., 2019; Schwarzer et al., 2020; Hafner et al., 2023; Watter et al., 2015; Nair et al., 2022; Zhang et al., 2022; Bagatella et al., 2026), enabling efficient learning and planning from pixels in both simulation and real-world robotics.

Recent work further demonstrates that task-agnostic generative world models trained on large-scale, real-world video can produce realistic and diverse simulations of physical environments (Hu et al., 2023; Yang et al., 2023; Bruce et al., 2024; Bai et al., 2025), highlighting the potential of scaling predictive modeling beyond task-specific datasets. Closely related to our setting, a line of work on latent world models shows that models trained on reward-free offline data via next-latent prediction can generalize zero-shot to downstream control tasks. These approaches leverage the learned dynamics to simulate action outcomes and select actions through cost-based optimization, without requiring task-specific policy training (Assran et al., 2025; Sobal et al., 2025; Terver et al., 2026b; Zhou et al., 2024).

Limitations of Long Horizon Planning. Learned world models remain brittle when used for long-horizon planning due to compounding prediction errors (Talvitie, 2014; Janner et al., 2019). Moreover, long-horizon planning is challenging even under a perfect model because the search tree grows exponentially with the branching factor

and horizon (a manifestation of the curse of horizon) (Ichter et al., 2020).

Hierarchical Reinforcement Learning. To address long-horizon control, prior work has explored hierarchical reinforcement learning through temporal abstraction via options and skills (Sutton et al., 1999; Bacon et al., 2017; Hafner et al., 2022). In parallel, many model-based RL approaches incorporate hierarchical world models to capture dynamics at multiple temporal resolutions (Gumbsch et al., 2023; Chitnis et al., 2024; Schiewer et al., 2024; Hansen et al., 2024). While these methods enable improved long-horizon reasoning, they typically rely on task-specific policy learning, which limits their applicability to zero-shot and general-purpose control.

Hierarchical Model Predictive Control. In the optimal control literature, hierarchical model predictive control (MPC) has been explored for long-horizon, multi-task settings (Fang et al., 2019; Li et al., 2021; Kögel et al., 2025). However, existing hierarchical MPC approaches are largely restricted to low-dimensional state spaces, hand-engineered representations, or known dynamics, limiting their applicability to high-dimensional observation spaces such as raw pixels.

We provide a more detailed comparison of these approaches, including their relationship to our method and a summary across key design axes, in [appendix A](#).

6 Conclusion

We presented HWM, a hierarchical planning framework that performs model predictive control over learned latent dynamics at multiple temporal scales. By decoupling long-horizon reasoning from fine-grained action execution, HWM mitigates compounding prediction errors and the exponentially growing search spaces that limit single-level world-model-based planners. Across diverse world-model architectures, hierarchical planning consistently outperforms flat planners on non-greedy, long-horizon tasks while substantially improving inference-time efficiency.

Despite these gains, long-horizon manipulation remains challenging, and performance degrades for all methods as horizons increase. Addressing this limitation will likely require advances in representation learning, including more abstract representations for higher-level world models; uncertainty-aware planning to better handle the increased stochasticity and model uncertainty inherent to long-horizon real-world prediction; and improved hierarchical planning algorithms that allow feedback and interaction across levels of the hierarchy instead of strictly top-down decomposition. We view HWM as a step toward scalable, general-purpose planning with learned world models, and hope this work encourages future research on hierarchical abstractions for long-horizon decision making in complex environments.

References

- Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- Pranav Atreya, Karl Pertsch, Tony Lee, Moo Jin Kim, Arhan Jain, Artur Kuramshin, Clemens Eppner, Cyrus Neary, Edward Hu, Fabio Ramos, et al. Roboarena: Distributed real-world evaluation of generalist robot policies. In *Proceedings of the Conference on Robot Learning (CoRL 2025)*, 2025.
- Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31, 2017.
- Marco Bagatella, Matteo Pirodda, Ahmed Touati, Alessandro Lazaric, and Andrea Tirinzoni. TD-JEPA: Latent-predictive representations for zero-shot reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026. <https://openreview.net/forum?id=SzXDUBN8M1>.
- Yutong Bai, Danny Tran, Amir Bar, Yann LeCun, Trevor Darrell, and Jitendra Malik. Whole-body conditioned egocentric video prediction. *arXiv preprint arXiv:2506.21552*, 2025.
- Amir Bar, Gaoyue Zhou, Danny Tran, Trevor Darrell, and Yann LeCun. Navigation world models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 15791–15801, 2025.
- Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. *arXiv preprint arXiv:2105.04906*, 2021.
- Jake Bruce, Michael D Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, et al. Genie: Generative interactive environments. In *Forty-first International Conference on Machine Learning*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- Rohan Chitnis, Yingchen Xu, Bobak Hashemi, Lucas Lehnert, Urun Dogan, Zheqing Zhu, and Olivier Delalleau. Iql-td-mpc: Implicit q-learning for hierarchical model predictive control. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9154–9160. IEEE, 2024.
- Marc Deisenroth and Carl E Rasmussen. Pilco: A model-based and data-efficient approach to policy search. In *Proceedings of the 28th International Conference on machine learning (ICML-11)*, pages 465–472, 2011.
- Frederik Ebert, Chelsea Finn, Alex X Lee, and Sergey Levine. Self-supervised visual planning with temporal skip connections. *CoRL*, 12(16):23, 2017.
- Kuan Fang, Yuke Zhu, Animesh Garg, Silvio Savarese, and Li Fei-Fei. Dynamics learning with cascaded variational inference for multi-step manipulation. *arXiv preprint arXiv:1910.13395*, 2019.
- Christian Gumbsch, Noor Sajid, Georg Martius, and Martin V Butz. Learning hierarchical world models with adaptive temporal abstractions from discrete latent dynamics. In *The Twelfth International Conference on Learning Representations*, 2023.
- David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3), 2018.
- Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pages 2555–2565. PMLR, 2019.
- Danijar Hafner, Kuang-Huei Lee, Ian Fischer, and Pieter Abbeel. Deep hierarchical planning from pixels. *Advances in Neural Information Processing Systems*, 35:26091–26104, 2022.
- Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.
- Nicklas Hansen, Hao Su, and Xiaolong Wang. Td-mpc2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2023.
- Nicklas Hansen, Jyothir SV, Vlad Sobal, Yann LeCun, Xiaolong Wang, and Hao Su. Hierarchical world models as visual whole-body humanoid controllers. *arXiv preprint arXiv:2405.18418*, 2024.

- Anthony Hu, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. Gaia-1: A generative world model for autonomous driving. *arXiv preprint arXiv:2309.17080*, 2023.
- Brian Ichter, Pierre Sermanet, and Corey Lynch. Broadly-exploring, local-policy trees for long-horizon task planning. *arXiv preprint arXiv:2010.06491*, 2020.
- Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- Michael Janner, Justin Fu, Marvin Zhang, and Sergey Levine. When to trust your model: Model-based policy optimization. *Advances in neural information processing systems*, 32, 2019.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- Markus Kögel, Mohamed Ibrahim, Christian Kallies, and Rolf Findeisen. Safe hierarchical model predictive control and planning for autonomous systems. *International Journal of Robust and Nonlinear Control*, 35(7):2658–2676, 2025.
- Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit q-learning. *arXiv preprint arXiv:2110.06169*, 2021.
- Vikash Kumar, Rutav Shah, Gaoyue Zhou, Vincent Moens, Vittorio Caggiano, Abhishek Gupta, and Aravind Rajeswaran. Robohive: A unified framework for robot learning. *Advances in Neural Information Processing Systems*, 36:44323–44340, 2023.
- Tianyu Li, Roberto Calandra, Deepak Pathak, Yuandong Tian, Franziska Meier, and Akshara Rai. Planning in learned latent action spaces for generalizable legged locomotion. *IEEE Robotics and Automation Letters*, 6(2):2682–2689, 2021.
- Lucas Maes, Quentin Le Lidec, Dan Haramati, Nassim Massaudi, Damien Scieur, Yann LeCun, and Randall Balestriero. stable-worldmodel-v1: Reproducible world modeling research and evaluation. *arXiv preprint arXiv:2602.08968*, 2026.
- Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3m: A universal visual representation for robot manipulation. *arXiv preprint arXiv:2203.12601*, 2022.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- Seohong Park, Dibya Ghosh, Benjamin Eysenbach, and Sergey Levine. Hiql: Offline goal-conditioned rl with latent states as actions. *Advances in Neural Information Processing Systems*, 36:34866–34891, 2023.
- Seohong Park, Kevin Frans, Benjamin Eysenbach, and Sergey Levine. Ogbench: Benchmarking offline goal-conditioned rl. *arXiv preprint arXiv:2410.20092*, 2024a.
- Seohong Park, Tobias Kreiman, and Sergey Levine. Foundation policies with hilbert representations. *arXiv preprint arXiv:2402.15567*, 2024b.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Reuven Y Rubinstein and Dirk P Kroese. *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation and machine learning*. Springer Science & Business Media, 2004.
- Robin Schiewer, Anand Subramoney, and Laurenz Wiskott. Exploring the limits of hierarchical world models in reinforcement learning. *Scientific Reports*, 14(1):26856, 2024.
- Max Schwarzer, Ankesh Anand, Rishab Goel, R Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. *arXiv preprint arXiv:2007.05929*, 2020.
- Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim GJ Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models. *arXiv preprint arXiv:2502.14819*, 2025.
- Richard S Sutton. An adaptive network that constructs and uses an internal model of its world. *Cognition and Brain Theory*, 4(3):217–246, 1981.

- Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM Sigart Bulletin*, 2(4): 160–163, 1991.
- Richard S Sutton, Doina Precup, and Satinder Singh. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211, 1999.
- Erik Talvitie. Model regularization for stable sample rollouts. In *UAI*, pages 780–789, 2014.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Basile Terver, Randall Balestriero, Megi Dervishi, David Fan, Quentin Garrido, Tushar Nagarajan, Koustuv Sinha, Wancong Zhang, Mike Rabbat, Yann LeCun, et al. A lightweight library for energy-based joint-embedding predictive architectures. *arXiv preprint arXiv:2602.03604*, 2026a.
- Basile Terver, Tsung-Yen Yang, Jean Ponce, Adrien Bardes, and Yann LeCun. What drives success in physical planning with joint-embedding predictive world models?, 2026b. <https://arxiv.org/abs/2512.24497>.
- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *IROS*, pages 5026–5033. IEEE, 2012. ISBN 978-1-4673-1737-5. <http://dblp.uni-trier.de/db/conf/iros/iros2012.html#TodorovET12>.
- Manuel Watter, Jost Springenberg, Joshka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. *Advances in neural information processing systems*, 28, 2015.
- Grady Williams, Nolan Wagener, Brian Goldfain, Paul Drews, James M Rehg, Byron Boots, and Evangelos A Theodorou. Information theoretic mpc for model-based reinforcement learning. In *2017 IEEE international conference on robotics and automation (ICRA)*, pages 1714–1721. IEEE, 2017.
- Mengjiao Yang, Yilun Du, Kamyar Ghasemipour, Jonathan Tompson, Dale Schuurmans, and Pieter Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 1(2):6, 2023.
- Wancong Zhang, Anthony GX-Chen, Vlad Sobal, Yann LeCun, and Nicolas Carion. Light-weight probing of unsupervised representations for reinforcement learning. *arXiv preprint arXiv:2208.12345*, 2022.
- Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. Dino-wm: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

Appendix Contents

A Relationship to Prior Hierarchical Model-Based Methods	16
B Training Details	17
B.1 Low Level World Model	17
B.2 Franka Arm with Robotiq Gripper	17
B.3 Push-T	18
B.4 Diverse Maze	18
C Planning Details	19
D Computational Analysis Details	20
D.1 Push-T	20
D.2 Diverse Maze	21
E Visualizing World Model Predictions	21
F Implementations	22
G Robot Task Definitions	23
H Additional Rollouts for Franka	24
I Offline Planning Evaluation for Franka	25
J Baselines	25
J.1 Vision-Language-Action Models	25
J.2 Goal-Conditioned & Zero-Shot RL Methods	26

A Relationship to Prior Hierarchical Model-Based Methods

In this section, we clarify the distinctions between HWM and prior methods along key axes.

We compare methods along the following dimensions:

- **Observation space:** whether planning operates on structured state or learned representations from high-dimensional observations (e.g., pixels).
- **Training paradigm:** whether hierarchy is learned via policies/skills or introduced at inference time on pretrained models.
- **Hierarchical interface:** how high-level decisions are translated into low-level control.

Method	Zero-shot	Representation	Training	Hierarchical interface
HWM (ours)	✓	pixels $\rightarrow$ learned latent	Learned models	Subgoal matching (latent)
CAVIN (Fang et al., 2019)	✓	Structured state	Learned models	Subgoal matching (state)
LAT (Li et al., 2021)	✓	Structured state	Learned model	Skill conditioned policy
Safe HMPC (Kögel et al., 2025)	×	Structured state	Analytical dynamics	Reference tracking (state)
IQL-TD-MPC (Chitnis et al., 2024)	×	state $\rightarrow$ learned latent	Offline RL + MPC	Goal-conditioned policy
Director (Hafner et al., 2022)	×	pixels $\rightarrow$ learned latent	Model-based RL	Goal-conditioned policy
Puppeteer (Hansen et al., 2024)	×	pixels $\rightarrow$ learned latent	Model-based RL	Goal-conditioned policy
THICK (Gumbsch et al., 2023)	partial	pixels $\rightarrow$ learned latent	Learned Models + RL	Subgoal matching (latent) + reward maximization

Table 5 Comparison of hierarchical model-based methods across key design axes. We categorize methods by whether they support zero-shot planning, the observation space of the model, the training paradigm, and the hierarchical interface between levels. We denote “partial” zero-shot when a method generalizes within a fixed, reward-defined task distribution but remains fundamentally tied to reward supervision, and thus does not support task-agnostic planning. In contrast, HWM enables fully reward-free, task-agnostic zero-shot planning.

Table 5 highlights several important distinctions between HWM and prior hierarchical model-based approaches.

Hierarchical Model Predictive Control methods (e.g., CAVIN, LAT, Safe HMPC) typically operate in a zero-shot setting, but are largely restricted to low-dimensional or structured state representations, often with known or simplified dynamics. This makes it challenging to directly apply them to high-dimensional perceptual inputs.

Hierarchical Model-Based Reinforcement Learning methods (e.g., Director, Puppeteer, IQL-TD-MPC) address this limitation by learning dynamics in latent representation spaces derived from high-dimensional observations. However, these approaches rely on reward supervision and are typically trained for specific task objectives, which can limit their applicability in zero-shot settings.

In contrast, HWM operates on learned latent world models trained purely via next-latent prediction from high-dimensional observations, without reward supervision. This enables a task-agnostic formulation in which planning is performed directly in the learned latent space. Concretely, both levels of the hierarchy optimize a shared objective that minimizes the distance between predicted latent states and a goal latent representation, with high-level predictions serving as subgoals for the low-level planner (section 2.3). This allows the system to compose behaviors at test time and adapt to new goals and environments in a fully zero-shot manner.

Among prior work, THICK is most closely related to our setting, as it also learns hierarchical latent world models from pixel observations. However, it is developed within a reward-predictive model-based RL framework, where models are trained with reward signals and optimized for reward-based objectives. As a result, its generalization is primarily evaluated within a fixed task distribution. In addition, its MPC-based evaluation focuses on relatively simple simulated settings with aligned training and test distributions and fixed goals.

B Training Details

B.1 Low Level World Model

We train the low-level latent world model $P_\theta^{(1)}$ to approximate the true transition dynamics $p(s_{t+1} | s_t, a_t)$. Given a trajectory sequence $\tau = (s_1, a_1, s_2, a_2, \dots, s_T)$, we first encode the states independently to obtain a sequence of latent features $(z_k)_{k \in [T]}$. We feed the interleaved sequence of actions and latent features $(a_k, z_k)_{k \in [T]}$ into the world model to obtain a sequence of next state representation predictions $(\hat{z}_{k+1})_{k \in [T]}$. The teacher-forcing loss is computed as:

$$\begin{aligned} \hat{z}_{k+1} &:= P_\theta^{(1)}((a_t, z_t)_{t \leq k}) \\ \mathcal{L}_{\text{tf}}(\theta) &:= \frac{1}{T} \sum_{k=1}^T \|\hat{z}_{k+1} - z_{k+1}\|_1. \end{aligned} \quad (2)$$

To mitigate compounding prediction errors over multiple steps, we additionally compute a multi-step autoregressive rollout loss. Let $P_\theta^{(1)}(a_{k:j}; z_k)$ denote the final predicted state representation obtained by autoregressively running the world model with an action sequence $a_{k:T}$, starting from z_k . We denote the rollout loss with prediction horizon T as:

$$\mathcal{L}_{\text{roll}}(\theta) := \sum_{j=2}^T \|P_\theta^{(1)}(a_{1:j}, z_1) - z_{j+1}\|_1 \quad (3)$$

The overall training objective is given by a weighted sum of the two quantities:

$$L(\theta) := \gamma_{\text{tf}} * \mathcal{L}_{\text{tf}}(\theta) + \gamma_{\text{roll}} * \mathcal{L}_{\text{roll}}(\theta) \quad (4)$$

B.2 Franka Arm with Robotiq Gripper

Data. We use approximately 96 hours of unlabeled videos from the DROID dataset (Khazatsky et al., 2024) and 30 hours from RoboSet (Kumar et al., 2023). Both datasets consist of short video clips of manipulation trajectories collected from real robots. Observations include RGB images and end-effector proprioception; actions correspond to end-effector delta poses.

State, Actions, and Preprocessing. Videos are sampled at a spatial resolution of 256×256 and with a frame rate uniformly sampled from (3, 10) FPS for DROID and (1, 5) FPS for RoboSet. At timestep k , the state is defined as $s_k = (x_k, p_k)$, where x_k is the raw RGB observation and $p_k \in \mathbb{R}^7$ is the end-effector state expressed in the robot base frame. The first three dimensions of p_k correspond to Cartesian position, the next three correspond to orientation represented as extrinsic Euler angles, and the final dimension represents the gripper state. Actions are defined as differences in end-effector state between consecutive frames: $a_k = p_{k+1} - p_k$. We apply random resize-and-crop augmentations to video clips, with the aspect ratio sampled uniformly from (0.75, 1.33).

Low Level World Model. Following the setup of VJEP2-AC (Assran et al., 2025), we sample fixed-length trajectories of $T = 15$ timesteps, $\{(a_k, p_k, x_k)\}_{k \in [15]}$. Each image x_k is encoded using the pretrained, frozen ViT-g/16 backbone, producing latent feature maps $(z_k)_{k \in [15]}$. The world model is instantiated as a $\sim 300\text{M}$ -parameter ViT. It takes temporally interleaved sequences of latent features, end-effector states, and actions $\{(z_k, p_k, a_k)\}_{k \in [15]}$ and predicts the next-step latent representations $\{\hat{z}_{k+1}\}_{k \in [15]}$. We refer to the VJEP2 paper for full architectural details.

High-Level World Model. To train the high-level world model, we sample variable-length trajectory segments $\{(a_k, p_k, x_k)\}_{k \in T}$ spanning $(0.33, 4)$ seconds and select $N = 3$ waypoint indices per segment, with the middle waypoint sampled uniformly at random. The high-level world model shares the same ViT architecture as the low-level model, but is conditioned on latent macro-actions rather than primitive actions. Conditioned on the current and previous waypoints and latent actions, it is trained via teacher forcing to predict the next waypoint. Latent macro-actions are produced by a transformer-based action encoder A_ψ , where the CLS token is passed through an MLP head to obtain the latent action.

World Model	Epochs	Batch Size	γ_{tf}	γ_{roll}	context T
VJEP2-AC (Low-Level)	200	256	1.0	1.0	16
VJEP2-AC (High-Level)	120	768	1.0	0.0	3

Table 6 Training hyperparameters for Droid + Roboset. γ_{tf} and γ_{roll} denote the coefficients for teacher-forcing and rollout losses, respectively, and context T specifies the model context length. All other hyperparameters follow those used by VJEP2-AC.

B.3 Push-T

Data. We use the offline dataset from DINO-WM, which consists of 18500 trajectories. Each trajectory consists of a sequence of (x_t, a_t, p_t) , where x_t is an RGB observation, a_t is a 2D action vector, and p_t is the proprioceptive state denoting current agent position and velocity.

Low Level World Model. We use DINO-WM (Zhou et al., 2024) as our low-level world model. The encoder is a pretrained DINOv2 network (Oquab et al., 2023), which is kept frozen during training. The world model is a 25M params ViT with causal masking that attends to past actions and latent states $(a_k, z_k)_{k \in T}$ over a context window of $T = 5$. Each transition $k \rightarrow k + 1$ corresponds to a stride of 5 frames in the original dataset. As a result, DINO-WM models over an effective horizon of $5T = 25$ timesteps in the original trajectory. The action a_k is formed by concatenating the 5 primitive actions executed between k and $k + 1$, yielding a $5 \times 2 = 10$ -dimensional action vector. Causal attention is implemented at the patch level: each latent patch vector z_k^i attends only to its corresponding patches $z_{k-T:k-1}^i$ from previous timesteps. Actions are first embedded using an MLP and then concatenated to each patch vector z_k^i . Proprioceptive inputs are incorporated using the same embedding and concatenation scheme. We refer to the original DINO-WM paper for full architectural details.

High-Level World Model. To construct training sequences, we subsample trajectory segments with lengths uniformly drawn between 25 and 70 timesteps. From each segment, we sample $N = 5$ waypoint states, which define the high-level transitions. The high-level world model is a scaled-up variant of the low-level world model (25M $\rightarrow$ 75M parameters), obtained by increasing the number of layers from 6 to 10, the embedding dimension from 384 to 768, and the MLP dimension from 2048 to 3072, while reducing the number of attention heads from 16 to 12. This increased capacity enables more effective modeling of long-horizon dynamics. Latent macro-actions are produced by a transformer-based action encoder A_ψ , where the CLS token is passed through an MLP head to obtain the latent action representation. See table 7 for hyperparameters.

World Model	Epochs	Batch Size	γ_{tf}	γ_{roll}	context T
DINO-WM (Low-Level)	100	256	1.0	0.0	4
DINO-WM (High-Level)	500	128	1.0	0.0	5

Table 7 Training hyperparameters for Push-T. γ_{tf} and γ_{roll} denote the coefficients for teacher-forcing and rollout losses, respectively, and context T specifies the model context length. All other hyperparameters follow those used by DINO-WM on Push-T.

B.4 Diverse Maze

Environment. The environment is a variant of Mujoco PointMaze (Todorov et al., 2012), which contains a point mass agent with a 4D state vector (global x , global y , v_x , v_y), where v is the agent velocity. The model receives a top down view of the maze rendered as $(98, 98, 3)$ RGB image tensor as input, instead of relying on

(global x , global) directly. Mujoco PointMaze allows for customization of the maze layout via a grid structure, where a grid cell can either be a wall or space. We opt for a 8×8 grid (excluding outer wall). Maze layouts are generated randomly. Only the following constraints are enforced: 1) all the space cells are interconnected, 2) percentage of space cells range from 50% to 80%.

Data. Each episode is initialized by placing the agent at a random (x, y) location in the maze and sampling an initial velocity (v_x, v_y) . A trajectory is then generated by randomly sampling and executing actions for 100 steps, using an action repeat of 4. We collect 2,000 episodes per map and use 25 maps for training; resulting in a total of 5M transitions.

Low Level World Model. We train the low-level world model following the PLDM recipe (Sobal et al., 2025), jointly learning the encoder and world model from scratch using offline trajectories. Each training sample consists of a sequence of 15 timesteps of observations and actions subsampled from a trajectory. The observation sequence is encoded by a convolutional network into spatial latent features $z_{1:15}$. Conditioned on the initial latent z_1 and the action sequence $a_{1:15}$, a convolutional world model autoregressively predicts the future latent sequence $\hat{z}_{1:15}$. To prevent representational collapse, we apply VICReg (Bardes et al., 2021) to the latent features $z_{1:15}$, following the original PLDM formulation. Refer to table 8 for hyperparameters. We use the same PLDM architecture for Diverse Maze, comprising a lightweight convolutional encoder (33k params) and predictor (20k params); we refer to the original paper for full details.

High-Level World Model. To construct training sequences, we subsample 60 timesteps from each trajectory and extract 6 waypoint states using a fixed stride of 10 timesteps. Primitive action chunks are encoded into an 8-dimensional latent action via an MLP. We reuse the backbone of a pretrained low-level world model and keep it frozen, training only the high-level world model. The high-level model is a higher-capacity convolutional network, enabling long-horizon prediction. Conditioned on the initial waypoint latent z_1 and the latent action sequence $l_{1:6}$, it autoregressively predicts subsequent waypoint latents. Hyperparameters are provided in table 8.

World Model	Epochs	Batch Size	lr	α	β	λ	ω	γ_{tf}	γ_{roll}	pred T	$\mathcal{L}_{proprio}$ coef.
PLDM (Low-Level)	3	128	0.018	29.4	17.9	2.80	4.81	0.0	1.0	15	2.42
PLDM (High-Level)	5	128	0.018	–	–	–	–	0.0	1.0	6	1

Table 8 Training hyperparameters for Diverse Maze. α , β , λ , and ω are parameters in the PLDM objective used to prevent representation collapse. $\mathcal{L}_{proprio}$ coef. denotes the coefficient on the proprioceptive prediction loss. γ_{tf} and γ_{roll} are the coefficients for teacher-forcing and rollout losses, and pred T is the number of prediction rollouts during training. For the high-level world model, we reuse the encoder trained at the low level and keep it frozen, eliminating additional representation learning.

C Planning Details

For Franka and Push-T, we use the Cross-Entropy Method (CEM) (Rubinstein and Kroese, 2004) for trajectory optimization in continuous action spaces. At each planning step, CEM maintains a factorized Gaussian over action sequences, iteratively sampling trajectories, evaluating them with the world model, and refitting to elite samples by updating the mean and variance. To improve stability and avoid premature collapse, variance updates are smoothed with an exponential moving average. After a fixed number of iterations, the optimized mean action sequence is executed, and planning proceeds in a receding-horizon manner. See table 9 and table 10 for the single-level and hierarchical planner parameters, respectively.

For Diverse Maze, we use Model Predictive Path Integral (MPPI) (Williams et al., 2017) for planning. MPPI updates a nominal action sequence by importance-weighting sampled trajectories using exponential costs; unlike CEM, it applies soft weights over all samples, yielding smoother updates without hard elite selection. We use the MPPI implementation from PLDM¹. See table 11 and table 12 for the single-level and hierarchical planner hyperparameters, respectively.

¹<https://github.com/vladisai/PLDM>

	# elites	# iters	# samples	Var EMA	pred h	k
Franka	20	15	2400	0.75	6	1
Push-T ($d = 25$)	30	30	300	0	5	5
Push-T ($d = 50$)	10	30	1200	0.9	10	5
Push-T ($d = 75$)	20	20	1000	0	15	5

Table 9 Single-level World Model Planning CEM hyperparameters for Franka and Push-T tasks. Pred h indicates prediction horizon.

	High-level planner					Low-level planner					
	# elites	# iters	# samples	Var EMA	pred H	# elites	# iters	# samples	Var EMA	pred h	k
Franka	22	15	3000	0.65	2	12	5	800	0.25	2	1
Push-T($d = 25$)	10	20	900	0	2	10	30	300	0	5	5
Push-T($d = 50$)	10	40	1500	0.9	4	10	20	900	0.8	5	5
Push-T($d = 75$)	10	20	1200	0.9	5	10	20	1200	0.8	5	5

Table 10 Hierarchical Planning CEM hyperparameters for Franka and Push-T tasks. Pred H and h indicate prediction horizons for the high-level and low-level planners respectively.

	noise σ	# samples	λ	pred h	k
Maze ($D \in [5, 8]$)	5	125	0.0025	150	4
Maze ($D \in [9, 12]$)	5	250	0.0025	200	4
Maze ($D \in [13, 16]$)	5	250	0.0025	250	4

Table 11 Single-level World Model Planning MPPI hyperparameters for Diverse Maze tasks. Pred h indicates prediction horizon.

	High-level planner				Low-level planner				
	noise σ	# samples	λ	pred H	noise σ	# samples	λ	pred h	k
Maze ($D \in [5, 8]$)	10	2000	0.0025	25	5	500	0.0025	15	4
Maze ($D \in [9, 12]$)	10	2000	0.0025	35	5	500	0.0025	15	4
Maze ($D \in [13, 16]$)	10	4000	0.0025	47	5	1000	0.0025	15	4

Table 12 Hierarchical Planning MPPI hyperparameters for Diverse Maze tasks. Pred H and h indicate prediction horizons for the high-level and low-level planners respectively.

D Computational Analysis Details

D.1 Push-T

For the compute scaling analysis of Fig. 5 and Fig. 8, we performed a grid search over the key CEM hyperparameters of both high-level and low-level planners.

For $d = 50$ in Fig. 5, for the high level planner, we swept $\#samples \in [150, 300, 600, 900, 1200, 1500]$, $\#iters \in [10, 20, 30, 40]$, and Var EMA $\in [0.4, 0.7, 0.9]$. For the low-level planner (both when used standalone and as the low-level planner in the hierarchical setting), we swept $\#samples \in [150, 300, 600, 900, 1200]$, $\#iters \in [10, 20, 30, 40]$, and Var EMA $\in [0, 0.4, 0.7, 0.8, 0.9]$.

For $d = 75$, for the high-level planner, we swept the number of samples $\#samples \in [150, 300, 600, 900, 1200, 1500]$, the number of CEM iterations $\#iters \in [10, 20, 30, 40]$, and the standard deviation momentum coefficient Var EMA $\in [0.4, 0.7, 0.9]$. For the low-level planner, we swept $\#samples \in [150, 300, 600, 900, 1000, 1100, 1200, 1500]$, $\#iters \in [10, 20, 25, 30, 35, 40]$, and Var EMA $\in [0, 0.4, 0.5, 0.7, 0.8, 0.9, 0.95]$.

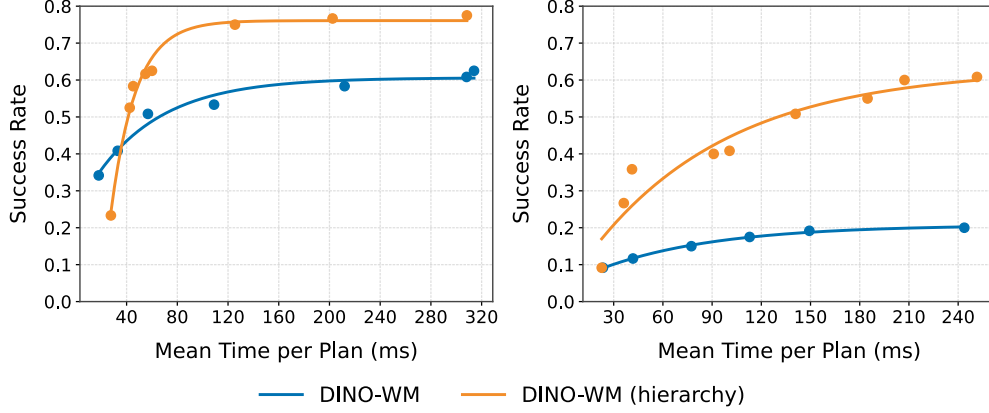


Figure 8 Success rate as a function of test-time compute for Push-T. **Left:** Setting $d = 50$. **Right:** Setting $d = 75$.

This comprehensive sweep over CEM hyperparameters enables the construction of Pareto frontiers that characterize the trade-off between planning compute (measured as mean time per plan) and task success rate.

D.2 Diverse Maze

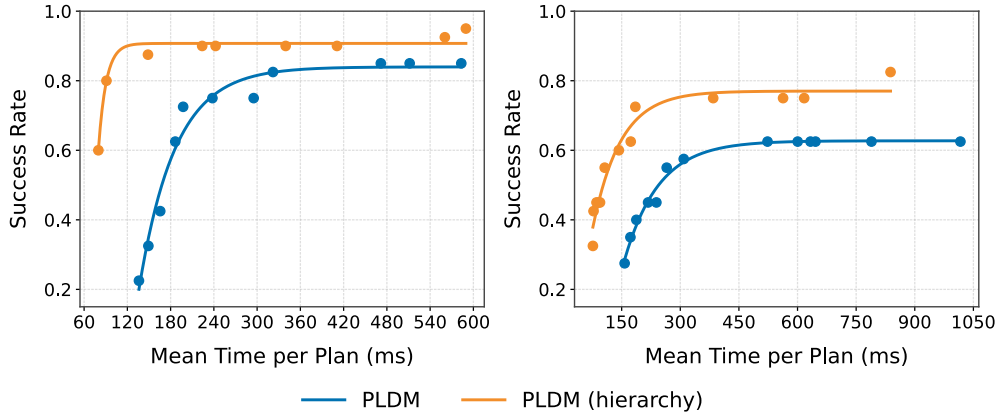


Figure 9 Success rate as a function of test-time compute for Diverse Maze. **Left:** Setting $D \in [9, 12]$. **Right:** Setting $D \in [13, 16]$.

For the compute scaling analysis of the Diverse Maze environment in Fig. 5 and 9, we performed a grid search over the number of samples and horizon of planning in the MPPI algorithm. We perform the same sweep for all 3 difficulty settings.

For the flat baseline, we swept the number of samples $\#\text{samples} \in [70, 125, 250, 500, 1000, 2000, 4000]$ and the horizon $\in [100, 125, 150, 175, 200, 225, 250, 275]$.

For the hierarchical planner, we have a low-level planner and a high-level planner, each of which has a number of samples. We swept the number of samples of the low-level planner $\#\text{samples} \in [70, 125, 250, 500, 1000, 2000, 4000]$, the number of samples of the high-level planner $\#\text{samples} \in [70, 125, 250, 500, 1000, 2000, 4000]$ and the horizon of the high-level planner $H \in [25, 35, 45]$. This comprehensive sweep over MPPI hyperparameters enables the construction of Pareto frontiers that characterize the trade-off between planning compute time and the success rate.

E Visualizing World Model Predictions

Franka Arm with Robotiq Gripper. To reconstruct RGB images from latent representations, we train on DROID a ViT decoder that maps the frozen V-JEPA 2 encoder features back to pixel space. The decoder takes as input the encoded features of dimension 1408 (matching the V-JEPA 2 ViT-Giant encoder output) and projects them

to an intermediate dimension of 1024 via a linear layer. The architecture consists of 12 transformer blocks, each with 32 attention heads and an MLP ratio of 4, using GELU activations, LayerNorm and sinusoidal 2D positional embeddings. The decoder operates on a 32×32 grid of spatial tokens (corresponding to patch size 8 for 256×256 images). The model is trained using an L_1 reconstruction loss on normalized pixel values. We train the decoder for 100 epochs on the DROID dataset using the AdamW optimizer with a peak learning rate of 5×10^{-4} , linear warmup over 2 epochs, cosine decay to 10^{-6} , and weight decay of 0.1. Training is performed on 8 nodes with a total batch size of 512.

Push-T. Similarly as for the DROID decoder depicted above, we train a ViT decoder that maps the frozen DINOv2 ViT-S/14 encoder features back to pixel space on the Push-T dataset. The decoder is a ViT-Base of intermediate dimension of 768 with 12 transformer blocks, each with 16 attention heads. It operates on a 28×28 grid of spatial tokens (corresponding to patch size 8 for 224×224 images). The model is trained using an L_2 reconstruction loss on normalized pixel values for 40 epochs on the PushT dataset with a total batch size of 256.

Diverse Maze. To visualize predictions and plans, we train a lightweight convolutional prober that takes as input a spatial representation of shape (H, W, C) . The prober consists of two convolution–pooling stages followed by a fully connected layer, and outputs 2-D coordinates corresponding to the agent’s (x, y) location. We show visualizations of successful high-level plans in Fig. 10.

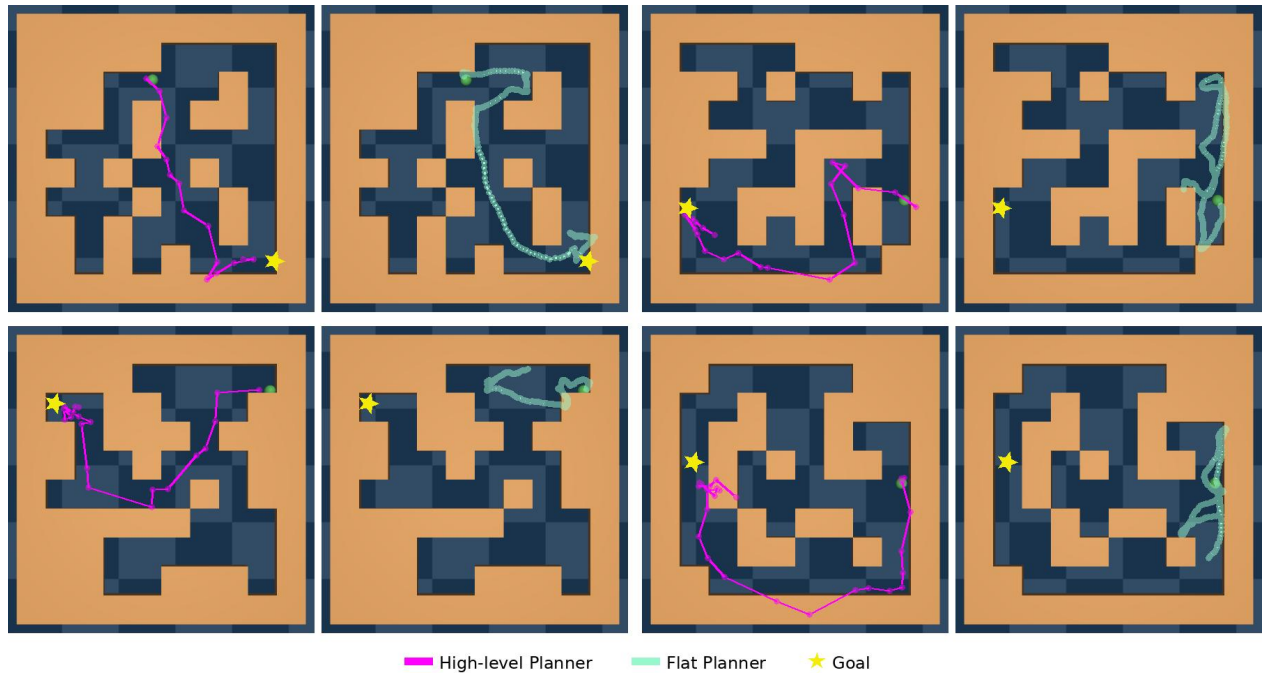


Figure 10 High-level plans in Diverse Maze. The agent is shown as a green ball and the goal as a yellow star. The pink trajectory is generated by the high-level planner, while the teal trajectory is generated by the flat planner. In these trials, the high-level planner successfully produces a sequence of subgoals that lead to the goal, whereas the flat planner often struggles due to the increased search complexity of long-horizon prediction.

F Implementations

We implement HWM directly on top the repositories of the following works:

VJEPa2 (Assran et al., 2025) for Franka experiments, PLDM (Sobal et al., 2025) for Diverse Maze, DINO-WM (Zhou et al., 2024) for Push-T.

Additional repositories for latent action-conditioned world-model based planning include eb-jepa and stable-worldmodel (Terver et al., 2026a; Maes et al., 2026).

G Robot Task Definitions

Fig. 11 depicts the initial and goal states defining the evaluation episodes for the Pick-&-Place Cup and Drawer tasks. We omit the Pick-&-Place Box tasks, which use identical start and goal configurations and differ only in the grasped object (a box rather than a cup).

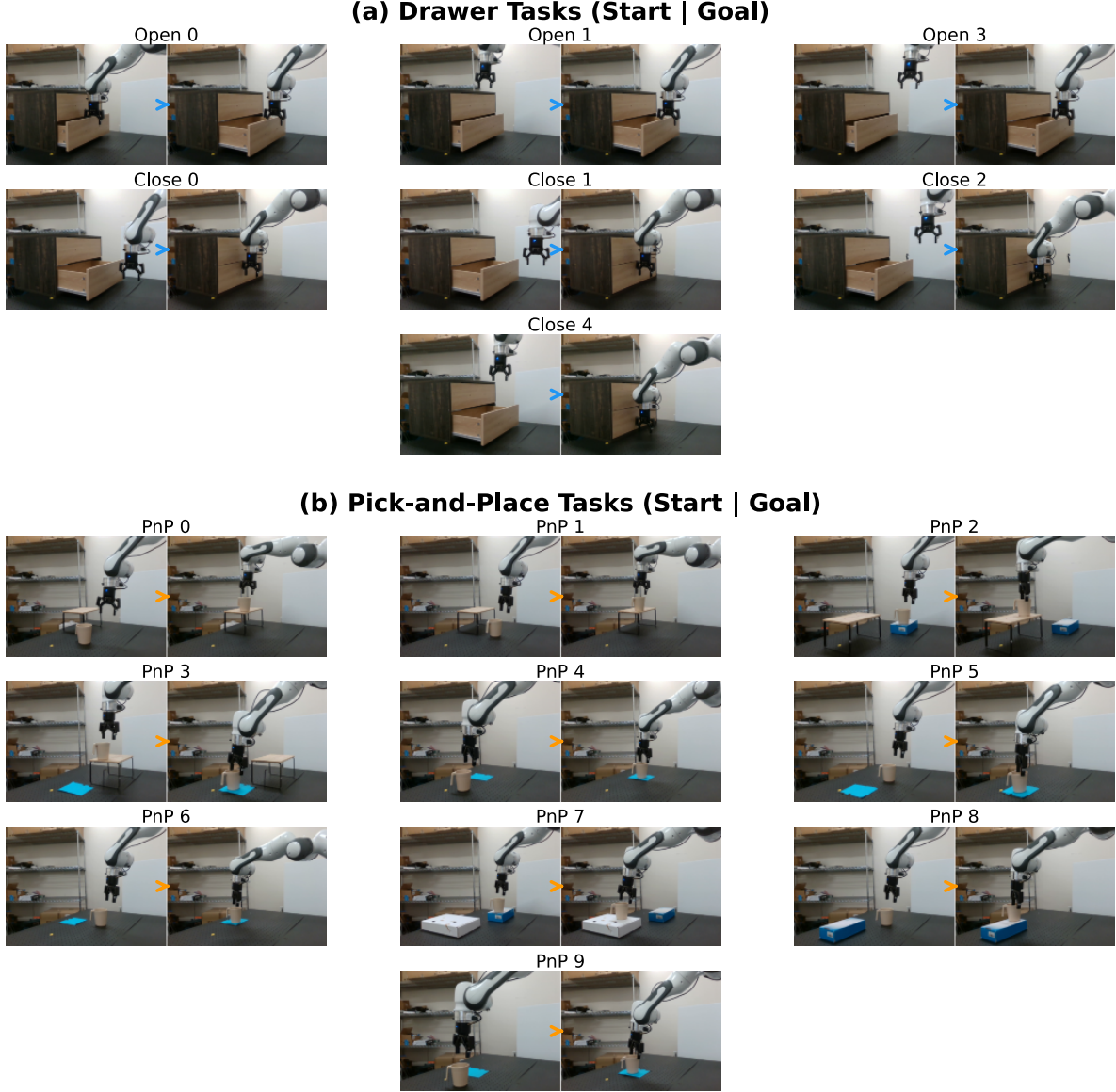


Figure 11 Initial and goal states for all evaluation tasks. **(a) Drawer Tasks:** 7 subtasks involving opening and closing drawers at various positions. **(b) Pick-&-Place Cup Tasks:** 10 subtasks requiring the robot to pick up a cup and place it at different target locations. Each panel shows the initial state (left) and goal state (right) for a given task.

H Additional Rollouts for Franka

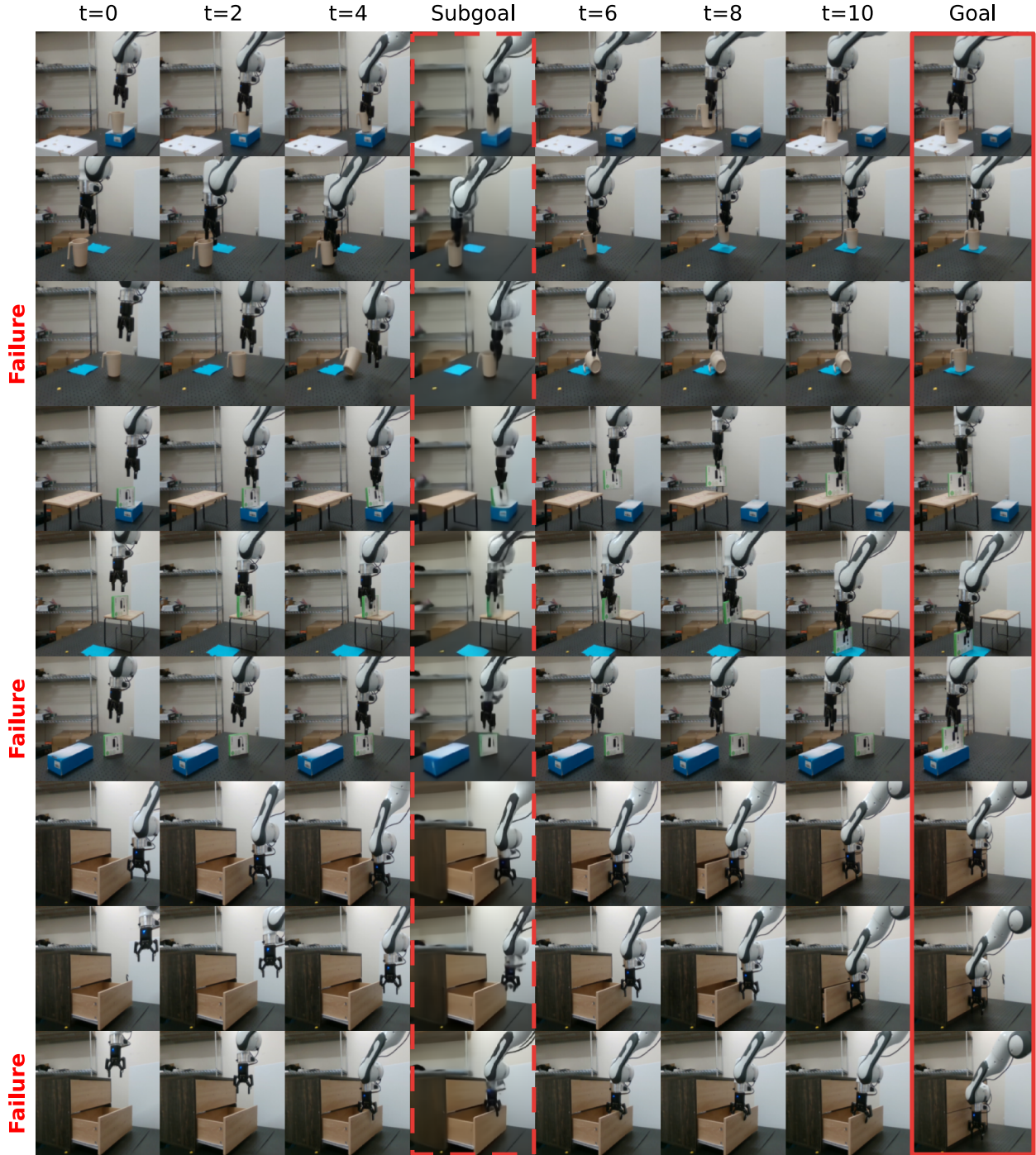


Figure 12 Executions on pick-&-place and drawer manipulation with a dual-gripper Franka arm. Dotted red columns indicate subgoals inferred by the high-level planner and decoded for visualization. Rows representing failed episodes are labeled with "Failure".

I Offline Planning Evaluation for Franka

Since planning on a real robot is time-consuming and not parallelizable, we introduce a quantitative offline planning evaluation that enables faster iteration over model variants and planning hyperparameters. We evaluate on the same 10 pick-&-place tasks used in the interactive experiments, with manually defined subgoals corresponding to the grasp point. Expert actions are defined as the robot’s delta pose from the initial observation to the subgoal, $\tilde{a}_1$, and from the subgoal to the goal, $\tilde{a}_2$. Conditioned on the initial and goal observations, the hierarchical planner produces a sequence of subgoals along with a sequence of primitive actions ($a_{1:h}^*$) toward the first subgoal. To assess high-level planning quality, we compare the aggregate inferred action $\sum_{i=1}^h a_i^*$ with the expert action $\tilde{a}_1$ using cosine similarity and ℓ_1 distance. High cosine similarity or low ℓ_1 distance indicates that the planner has identified a plan that moves toward the non-greedy subgoal, which strongly correlates with real-world task success.

J Baselines

J.1 Vision-Language-Action Models

For our baselines, we reproduce the same setup as in the Droid platform. Specifically, we use a Franka Panda arm with a bi-finger Robotiq gripper, dual camera setup (one wrist camera plus one side camera looking at the robot from the left). Our wrist camera setup is identical to Droid (the same mount and model - ZED Mini), however, for the side camera, we use Intel RealSense. While RealSense has a different aspect ratio (4:3 instead of ZED Mini’s 16:9) we found that simply padding the side image to get 16:9 aspect ratio yields good success rate for VLAs trained on Droid with this aspect ratio. Note that we use the same RealSense side camera for all baselines and world model-based agents therefore, they all have access to the same amount of sensor signal from the side view. In fact, the π - family of models receive extra sensor information since they have access to wrist camera which is not used by Octo and world model-based agents.

π_0 -FAST-DROID and $\pi_{0.5}$ -DROID (Pertsch et al., 2025; Intelligence et al., 2025). For a reliable control with the π_x -DROID models family, we reproduce the same controller on our Franka as in the Droid benchmark. Both models use one side camera and one wrist camera and output chunks of actions being joint velocities and target gripper positions. For both models, we execute the entire chunk of outputted actions before re-querying the policy (15 actions for $\pi_{0.5}$ and 8 actions for π_0 -FAST). We run both policies for 300 total actions as we found this to be enough for a successful task attempt with a big space for suboptimality. We do not finetune these policies, therefore, the evaluation is zero-shot and out-of-distribution since they are only finetuned on Droid so our scene is new to both of them.

Since π - models use text goal specification, we tuned text prompts to achieve the highest success rate. We arrived at the same conclusions as in the RoboArena (Atreya et al., 2025) benchmark, specifically, π - models do not understand colors and complex nouns for shapes (e.g. platform, shelf etc) and each prompt has to be written in a short and a simple way. Below are the prompts we tried (either “cup” or only “box” is present in the prompt):

1. “put the cup on the plane”
2. “put the cup (box) on the platform”
3. “put the cup (box) on the table” – the model was picking the object up and putting it on the the same position since the “ground level” surface is a table despite a small table was in front of it (See Figure 11).
4. “put the cup (box) on the yellow table” – same as 2.
5. “put the cup on the wooden table” – same as 2.
6. “put the cup on the shelf” – the policy tried to place the object on the shelf in the background.
7. “put the cup on the plate” – this prompt worked. Since it requires a plate, we started to use the one as the marker for the goal position for the object placement. *We believe makes language goals a good*

substitution for image goals as the plate position unambiguously informs the model where the object should be placed and the plate is present in many training episodes.

We run $\pi_{0.5}$ -DROID for 100 trials for the pick-&-place task with the cup and the pick-&-place task with the box. We use the same 10 task configurations as the ones present in Figure 11 to ensure fair comparison. This means we run 10 trials for each configuration which gives a low variance for the policy with highly stochastic outputs. Similarly, we run 100 trials for the pick-&-place task with the cup and 50 trials for the pick-&-place task with the box (i.e. 5 trials per configuration).

Similarly to the evaluation of the world models and policies, we use binary success rate for evaluations. In particular, we simply give the policy a score of 1 if the object was placed in the target position and zero otherwise. Unfortunately, we were unable to run drawer tasks with π - models because during our evaluation of the drawer task with Octo, the drawer was physically destroyed by the robot into a non-operating state. Note that because of language goals (which can only be primitive), π - models cannot run the task with subgoals.

Octo (Team et al., 2024). We use the same setup for octo as in the V-JEPA 2 paper. Specifically, we use the Octo-base policy finetuned on Droid with image goals. We used the same scoring as for the π - models. Note that Octo is not able to work well without subgoals so we found its performance without subgoals to be 0.

J.2 Goal-Conditioned & Zero-Shot RL Methods

GCIQL (Park et al., 2023) – goal-conditioned version of Implicit Q-Learning (Kostrikov et al., 2021), a strong and widely-used method for offline RL.

HIQL (Park et al., 2023) – a hierarchical GCRL method which trains two policies: one to generate subgoals, and another one to reach the subgoals. Notably, both policies use the same value function.

HILP (Park et al., 2024b) – a method that learns state representations from the offline data such that the distance in the learned representation space is proportional to the number of steps between two states. A direction-conditioned policy is then learned to be able to move along any specified direction in the latent space.

For GCIQL and HIQL, we use the implementations from the repository² of OGBench (Park et al., 2024a). Likewise, for HILP we use the official implementation³ from its authors. For Push-T and Diverse Maze, we tune hyperparameters independently for each difficulty setting and report results using the optimal configurations. The selected hyperparameters are reported in table 13; all other hyperparameters not listed in the table use the default values from the original repositories.

	GCIQL				HIQL				HILP		
	lr	expectile	rand goal	p	lr	expectile	rand goal	p	alpha	lr	expectile
Push-T($d = 25$)	3×10^{-5}	0.6	0.0		3×10^{-5}	0.9	0.0		3.0	3×10^{-5}	0.6
Push-T($d = 50$)	3×10^{-5}	0.6	0.3		3×10^{-5}	0.9	0.3		1.0	3×10^{-5}	0.6
Push-T($d = 75$)	3×10^{-5}	0.9	0.6		3×10^{-5}	0.9	0.3		1.0	3×10^{-5}	0.6
Maze($D \in [5, 8]$)	3×10^{-5}	0.8	0.0		3×10^{-5}	0.6	0.3		1.0	3×10^{-5}	0.9
Maze($D \in [9, 12]$)	3×10^{-5}	0.6	0.3		3×10^{-5}	0.8	0.6		3.0	3×10^{-5}	0.9
Maze($D \in [13, 16]$)	0.0001	0.6	0.0		3×10^{-5}	0.8	0.6		3.0	3×10^{-5}	0.9

Table 13 Hyperparameters. Hyperparameters used for GCIQL, HIQL, and HILP on the Push-T and Diverse Maze tasks.

²<https://github.com/seohongpark/ogbench>

³<https://github.com/seohongpark/HILP>